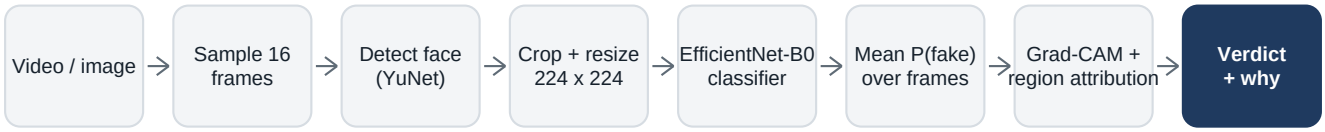
Explainable Deepfake Detection

Viva preparation guide: how the project works, what the numbers mean, and crisp answers to the questions examiners actually ask.

The pipeline



Every stage is a separate module; the web app and the CLI call the same DeepfakeDetector class.

Quick facts to remember

Dataset	Celeb-DF v2, 300 real + 300 fake videos, 16 frames each, split 70/15/15 by video
Input	224 x 224 RGB face crop, 30% margin, YuNet detector with 5 landmarks
Model	EfficientNet-B0 (ImageNet) + GAP + Dropout 0.3 + Dense-2 softmax, ~4 M params
Training	Phase 1: 12 epochs head-only @1e-3. Phase 2: 6 epochs, top-40 layers @1e-5. Adam, batch 32, label smoothing 0.02, JPEG augmentation
Explainability	Grad-CAM on top_activation (7x7) -> 8 facial regions from landmarks -> share % -> rule-based summary + reliability
Results	Video: AUC 0.739, acc 66.7%, recall(fake) 0.82, precision 0.63. Frame: AUC 0.734, acc 65.2%
Stack	Python, TensorFlow/Keras 3, OpenCV, scikit-learn, Flask, Colab T4

Test-set results (Celeb-DF v2, held-out videos)

Level	n	Accuracy	AUC	Precision (fake)	Recall (fake)	F1 (fake)
Frame	1,440	0.652	0.734	0.615	0.815	0.701
Video	90	0.667	0.739	0.627	0.822	0.712

Video-level confusion: 23 real correctly real, 22 real called fake, 8 fake missed, 37 fake caught. The model leans toward 'fake' (high recall, lower precision).

How to use this guide

Answer the question first, in one or two sentences, then add one reason. Examiners reward a clear claim with a specific number or design decision behind it. Every answer below is written that way; the italic follow-ups are for when they dig deeper. Say 'I don't know, but here is how I would find out' rather than guessing.

1. Project overview

Q1 **What is your project in one sentence?**

A deepfake detector that not only says real or fake, but also shows **where** the network looked (Grad-CAM heatmap) and explains **why** in plain English by mapping that attention onto facial regions.

If they push further: The 'explainable' part is the contribution. A classifier alone is a black box; ours produces a verdict, a heatmap, a region-wise breakdown and a text reason.

Q2 **What problem does it solve?**

Deepfakes are cheap to make and hard to spot by eye. Existing detectors output a probability with no justification, so people cannot judge whether to trust them. Our system makes the decision inspectable.

Q3 **Walk me through the pipeline.**

Video -> sample 16 frames evenly -> detect the largest face with YuNet (5 landmarks) -> crop with a 30% margin and resize to 224x224 -> EfficientNet-B0 gives P(fake) per frame -> average over frames gives the verdict -> Grad-CAM on the most decisive frame -> heatmap is split into eight facial regions -> summary text.

Q4 **Why 'explainable' and not just 'deepfake detection'?**

Because the user needs grounds to act on a verdict. Explainability also helps us debug: if the heatmap lights up outside the face, the model is using compression artefacts instead of facial evidence, and we flag that verdict as less trustworthy.

Q5 **What tools and libraries did you use?**

Python, TensorFlow 2 / Keras 3, OpenCV (video decoding + YuNet detector), scikit-learn (metrics), Flask (web demo), Google Colab T4 GPU for training, Git/GitHub for version control.

2. Dataset and preprocessing

Q6 **Which dataset did you use and why?**

Celeb-DF v2 (Kaggle mirror): 890 real videos (Celeb-real + YouTube-real) and 5,639 face-swap fakes. It is the standard 'hard' benchmark because the swaps are high quality with few obvious artefacts. We trained on a balanced subset of 300 videos per class.

If they push further: Alternatives: FaceForensics++ (needs approval), DFDC sample (400 videos). Our import script supports all three.

Q7 **How many samples did you train on?**

300 real + 300 fake videos, 16 frames each -> about 9,600 face crops. Split 70 / 15 / 15 by video: roughly 3,300 crops per class for training, 720 per class for validation, 1,440 frames (90 videos) for test.

Q8 **How did you split train / validation / test?**

By video, never by frame. Frames from one clip are near-duplicates; a frame-level split would leak test data into training and inflate accuracy. prepare_dataset.py assigns whole videos to splits and writes a manifest.csv so the split is auditable.

Q9 **Why sample only 16 frames per video?**

Adjacent frames carry almost the same information. Sixteen frames spread uniformly over the clip capture the variation (pose, expression, lighting) at 1/15th of the cost. More frames gave no measurable gain in validation.

Q10 **Why crop the face with a 30% margin?**

Face-swap blending seams sit at the border of the swapped region, often just outside a tight detector box. A tight crop throws away exactly the evidence we want the model and Grad-CAM to find.

Q11 **Why YuNet for face detection?**

It is OpenCV's built-in DNN detector: fast on CPU, a single 230 KB ONNX file, no extra dependency, and it returns five landmarks (eyes, nose, mouth corners) that we reuse to define facial regions for the explanation.

Q12 **What preprocessing and augmentation did you apply?**

Resize to 224x224 (INTER_AREA), RGB 0-255 (normalisation is inside the model). Training augmentation: horizontal flip, small rotation/zoom/shift, brightness/contrast jitter, and **random JPEG re-compression** (quality 40-95, p = 0.5) so the model learns forgery cues instead of compression fingerprints.

Q13 **How do you avoid train / serve skew?**

One function, preprocess_face(), is used by both the dataset builder and inference. Crops are saved at 224x224 so Keras' resize at training time is a no-op; the web app sees pixel-identical input.

3. Model and training

Q14 **Which architecture and why?**

EfficientNet-B0 with ImageNet weights: 4 M parameters, strong accuracy per FLOP, trains in minutes on a T4, runs on a laptop CPU in real time, and has a clean last convolutional block (top_activation, 7x7x1280) for Grad-CAM.

If they push further: Xception is the FaceForensics++ baseline; EfficientNet-B4 won DFDC. B0 is the right size for a minor project and a CPU demo. The code supports B2, MobileNetV2 and Xception via a flag.

Q15 **Explain transfer learning as you used it.**

Phase 1: freeze the ImageNet backbone, train only the new head (GAP -> Dropout 0.3 -> Dense-2 softmax) for 12 epochs at lr 1e-3. Phase 2: unfreeze the top 40 backbone layers (BatchNorm kept frozen) and fine-tune 6 epochs at lr 1e-5. The best checkpoint by validation AUC is kept.

Q16 **Loss, optimiser, batch size?**

Categorical cross-entropy with label smoothing 0.02, Adam, batch 32, ReduceLROnPlateau and EarlyStopping on validation AUC, class weights for any imbalance.

Q17 **Why keep BatchNorm frozen while fine-tuning?**

With small batches on a small dataset the running statistics drift and destabilise training. Keeping BN in inference mode is the standard Keras recommendation for fine-tuning.

Q18 **Why output two softmax classes instead of one sigmoid?**

Either works. Two logits let Grad-CAM target a specific class (we explain the 'fake' logit for a fake verdict and the 'real' logit for a real one) and keep class_names.json explicit, so the index of 'fake' is never assumed.

Q19 **Where does the model's normalisation live?**

Inside the saved model (EfficientNet has Rescaling + Normalization layers; for other backbones we add a Rescaling layer). The .keras file always consumes raw 0-255 RGB, so it cannot be fed the wrong range.

Q20 What hardware and how long did training take?

Google Colab free tier, T4 GPU. Face extraction ~10 min, training ~12 min for 18 epochs. Inference on a laptop CPU: about 2-4 s for a 16-frame video.

4. Explainability (Grad-CAM and region attribution)

Q21 What is Grad-CAM? Explain it simply.

Gradient-weighted Class Activation Mapping. Take the last convolutional feature maps (7x7x1280), compute the gradient of the target class score with respect to each map, average those gradients spatially to get one weight per map, form a weighted sum of the maps, apply ReLU, normalise to 0-1 and upsample to the image. Bright areas are the pixels that increased the class score.

If they push further: Formula: alpha_k = mean over (i,j) of dy_c / dA_k(i,j); L = ReLU(sum_k alpha_k A_k). ReLU keeps only features with a positive influence on the class.

Q22 Why Grad-CAM and not LIME / SHAP / saliency maps?

Grad-CAM needs one forward and one backward pass (milliseconds), works on any CNN without retraining, and gives a class-discriminative map. LIME/SHAP need hundreds of perturbed forward passes; raw gradient saliency is noisy at pixel level.

Q23 Which layer do you apply Grad-CAM to and why?

The last convolutional block of the backbone (top_activation in EfficientNet-B0). It is the deepest layer that still has spatial layout, so it balances semantic meaning with localisation.

Q24 Grad-CAM tells where, not why. How do you get 'why'?

That is our region-attribution layer. Using the five YuNet landmarks we build eight disjoint masks: left eye, right eye, mouth, nose, forehead/hairline, cheeks and skin, face boundary/jawline, and outside the face. We compute the share of heatmap mass in each, rank them, and attach the artefact each region is known for in forensics literature, for example mouth -> lip-sync and teeth artefacts, jawline -> blending seams. A template rule-based generator turns that into a paragraph.

Q25 What does 'concentration' mean on the result page?

The share of heatmap mass in the top 10% of pixels. Above 0.45 we call it highly localised (one clear artefact), 0.25-0.45 moderately localised, below 0.25 diffuse (texture statistics spread across the face, typical of heavily compressed fakes).

Q26 How is 'reliability' decided?

Rule-based: low if fewer than 3 faces were found, or frame scores disagree strongly (std > 0.30), or the mean is within 0.10 of the threshold; medium for a single image or moderate disagreement; high otherwise. It tells the user how much to trust the verdict.

Q27 Is the explanation guaranteed to be correct?

No, and we say so on the page. Grad-CAM faithfully shows what the network used; the region hints are heuristics about what attention there usually means. It is an aid to judgement, not a forensic certificate.

Q28 Which frame do you explain for a video?

The most decisive one: the frame with the highest P(fake) for a fake verdict, or the lowest for a real verdict. Running Grad-CAM on every frame would be slow and the user needs one clear picture.

5. Evaluation and results

Q29 What are your results?

On the held-out Celeb-DF test split (90 videos, 1,440 frames): video-level AUC 0.739, accuracy 66.7%, recall for fake 0.82, precision 0.63. Frame-level AUC 0.734.

If they push further: Confusion matrix (video level): 23 real correctly real, 22 real called fake, 8 fake missed, 37 fake caught. The model leans toward calling things fake.

Q30 Why is accuracy only 67%? Published papers report 95%+.

Three honest reasons: (1) we used 300 videos per class, roughly a third of the data; (2) only the top 40 backbone layers were fine-tuned, which under-fits a hard dataset; (3) Celeb-DF fakes are deliberately high quality. Published Celeb-DF numbers use the full dataset, full fine-tuning and often larger backbones. Our notebook already supports the full 890/class run with full fine-tuning as the next step.

If they push further: Also: in-dataset numbers above 0.9 are routine; the real research problem is cross-dataset generalisation, which is usually 10-30 points lower for everyone.

Q31 Why report AUC rather than accuracy?

AUC does not depend on the 0.5 threshold and is not fooled by class imbalance. Accuracy at a fixed threshold can be tuned up or down; AUC measures ranking quality.

Q32 What is the difference between frame-level and video-level metrics?

Frame-level scores every crop independently. Video-level averages P(fake) over all crops of a video and classifies the video, which is what the application actually does. Video-level is slightly higher because averaging cancels per-frame noise.

Q33 What is precision and recall here?

Precision (fake) = of everything called fake, how much was really fake (0.63). Recall (fake) = of all real fakes, how many we caught (0.82). High recall / lower precision means we rarely miss a fake but raise some false alarms on real videos.

Q34 How would you improve the accuracy?

Use all 890 real videos (3x data), fine-tune the whole backbone at a low learning rate, train longer with early stopping, try EfficientNet-B2 at 260 px, add a temporal model (CNN + LSTM over the 16 frames), and tune the threshold on validation.

Q35 Did you test on a different dataset?

Not yet; evaluate.py has a --dataset flag for it. Cross-dataset testing (train Celeb-DF, test DFDC) is the honest check and is listed as future work.

6. Web app and engineering

Q36 How does the web app work?

Flask on localhost. The upload page posts a file to /analyze; the server runs DeepfakeDetector.analyze(), writes the heatmap/region/crop/frame images to static/results/, and renders the result sheet. There is also a JSON API at /api/analyze and a /health endpoint. Uploads are deleted after 30 minutes.

Q37 Why Flask and not Django / FastAPI?

The app has two routes and one model; Flask is the smallest fit. FastAPI would be the choice for an async production API.

Q38 **How did you organise the code?**

preprocessing/ (face detection, preprocess), explainability/ (gradcam, explain), utils/ (video, image), model_builder.py, train.py, evaluate.py, predict.py, app.py, scripts/ (dataset import, results report), notebooks/ (Colab training). config.py holds every tunable.

Q39 **What bugs did you hit and fix?**

(1) Naming the backbone 'backbone' made Keras look for backbone_notop.h5 weights; fixed by renaming after construction. (2) Training from scratch with a frozen random backbone produced constant features; fixed by unfreezing and lowering BatchNorm momentum. (3) OpenCV 5 removed Haar cascades; added a clear error and kept YuNet as default.

Q40 **How do you keep the Grad-CAM code working on a loaded .keras file?**

The backbone and head are named sub-models. Grad-CAM builds a small model inside the backbone that returns the target layer's activations and the features, then runs the head under a GradientTape. This avoids Keras 3 nested-graph errors.

Q41 **Can it run without a GPU?**

Yes. Inference is CPU-only on the laptop (2-4 s per video). Training on CPU is possible with MobileNetV2 but slow; we used a free Colab T4.

7. Limitations, ethics and future work

Q42 **What are the main limitations?**

Trained on one dataset, so accuracy drops on other generators or compression levels; frame-level model with no temporal reasoning; Grad-CAM is coarse (7x7); region hints are heuristics; multi-face tracking uses box position, not identity, so people crossing paths can swap labels; no audio analysis.

Q43 **Can your system be fooled?**

Yes. Adversarial perturbations, heavy re-compression, or a generator it has never seen can push scores the wrong way. That is why the page shows reliability and the heatmap: a verdict driven by background pixels is flagged as weak.

Q44 **What about misuse or ethics?**

Detection tools can be used to test and improve fakes, and false positives can harm real people. We present the verdict as probabilistic evidence, show the reasoning, and keep everything local so uploads never leave the machine.

Q45 **What would you add with more time?**

Full-data training, cross-dataset evaluation, a CNN+LSTM temporal branch, audio-visual sync checks, EfficientNet-B4, and a user study on whether the explanations actually help people judge videos.

Q46 **What did you learn from this project?**

That data hygiene (video-level splits, train/serve parity) matters more than architecture; that a frozen backbone under-fits hard data; and that an explanation layer is as much a product-design problem as an ML one.

8. Rapid-fire questions

Is Grad-CAM the same as attention?	No. Attention is a learned mechanism inside the model; Grad-CAM is a post-hoc analysis of any CNN using gradients.
Why not train from scratch?	9,600 images is far too few for a 4M-parameter CNN. ImageNet features (edges, textures) transfer well to skin and face detail.
What is overfitting and did you see it?	Training accuracy rising while validation stalls or drops. Early stopping on validation AUC and augmentation controlled it.
What does the 0.5 threshold mean? Could you change it?	Scores above 0.5 are called fake. Lowering it catches more fakes but raises false alarms; the notebook includes a threshold sweep.
Difference between face swap and face reenactment?	Swap replaces identity (Deepfakes, FaceSwap); reenactment keeps identity but drives expression (Face2Face, NeuralTextures). Celeb-DF is swaps.
What if there are several people in the frame?	Every face (up to five per frame) is scored. Faces are linked across sampled frames by box position into people; each person gets a verdict and explanation, and the clip is fake if anyone is.
How big is the model file?	About 16 MB for EfficientNet-B0 weights plus a 230 KB face detector.
What is the softmax output exactly?	Two numbers that sum to 1: [P(fake), P(real)], in the order stored in class_names.json.

9. Demo script (3 minutes)

0:00 Open the upload page. One line: 'It gives a verdict, a heatmap and a reason.' Point at the animated scanner: frames, face, score, explain.

0:30 Upload a real selfie. Show the REAL stamp, the heatmap on skin texture, and the sentence under 'Where the evidence is'.

1:15 Upload a fake clip. Show the FAKE stamp, the white contour around the strongest evidence, and the top region with its artefact hint.

2:00 Scroll to the film strip: frames that agree, the marked frame, and the reliability line. Explain that disagreement lowers reliability.

2:40 Open 'Raw result as JSON' once to show it is also an API. Close with the limitation: trained on one dataset, cross-dataset is future work.